

Informe individual — Sesión 2

Archivo de entrega (PDF): [S02_Equipo01_CuadrosJhosef.pdf](#)

Universidad: Universidad Peruana Unión (UPeU) — Ingeniería de Sistemas

Proyecto: pagatu

Framework: Spring Boot 4.0.7 · Java 21 · Spring Cloud 2025.1.2 (Oakwood)

1. Datos del estudiante (estructura 4.3.1)

- **Nombre:** Cuadros Flores Jhosef Giampiere
 - **Equipo:** 1
 - **Sesión:** S02 - Gestión Centralizada de Configuración y Ambientes
 - **Rol o aporte realizado:** Desarrollo e integración autónoma del microservicio cliente (pagatu-orden-ms), migración a Config Client, creación de archivos .yaml en config-repo y validación de endpoints HTTP/Actuator.
 - **Link de GitHub:** [INSERTAR_LINK_AQUI]
-

2. Evidencia técnica

Las capturas deben cumplir el estándar de la guía: pantalla completa, sin recortes, con reloj del sistema (fecha y hora) y nombre de usuario de Windows / VS Code visibles.

2.1. Comandos de arranque (para reproducir la evidencia)

Desde la raíz del workspace:

```
docker compose -f docker-compose.s02.yml up -d
```

Config Server (el directorio de trabajo debe ser [infra/pagatu-config](#) para que `file:./config-repo` resuelva):

```
cd infra/pagatu-config
mvn spring-boot:run
```

Cliente autónomo [orden-ms](#) (perfil `dev` por defecto):

```
cd services/pagatu-orden-ms
mvn spring-boot:run
```

URLs usadas en esta sesión:

Recurso	URL
Config Server Health	http://localhost:18888/actuator/health
Config Server Metrics	http://localhost:18888/actuator/metrics
Environment orden-ms DEV	http://localhost:18888/orden-ms/dev
Environment orden-ms PROD	http://localhost:18888/orden-ms/prod
Environment catálogo DEV	http://localhost:18888/pagatu-catalogo-ms/dev
orden-ms Health	http://localhost:8082/actuator/health
orden-ms Info de configuración	http://localhost:8082/api/v1/ordenes/info

2.2. pagatu-config respondiendo en `/actuator/health` y `/actuator/metrics` (DEV)

Se levantó `pagatu-config` con perfil `native`, puerto **18888** y `search-locations: file:./config-repo`. Actuator expone `health`, `info` y `metrics`.

Health (DEV): se espera `status: UP` y el componente `configServer`.

```
curl http://localhost:18888/actuator/health
```

[INSERTAR CAPTURA DE PANTALLA AQUÍ: Debe mostrar la pantalla completa sin recortar, con el reloj del sistema (fecha y hora) y el nombre de usuario de Windows/Vs Code visible]

Métrica (DEV): se consulta el catálogo de métricas y al menos un meter (por ejemplo `jvm.memory.used`).

```
curl http://localhost:18888/actuator/metrics
curl http://localhost:18888/actuator/metrics/jvm.memory.used
```

[INSERTAR CAPTURA DE PANTALLA AQUÍ: Debe mostrar la pantalla completa sin recortar, con el reloj del sistema (fecha y hora) y el nombre de usuario de Windows/Vs Code visible]

2.3. Consultas HTTP a `/orden-ms/dev` y `/orden-ms/prod` (JSON de Environment)

El Config Server publica el `Environment` con la convención `{application}/{profile}`:

```
curl http://localhost:18888/orden-ms/dev
curl http://localhost:18888/orden-ms/prod
```

En DEV el JSON debe incluir `name: orden-ms`, `profiles: [dev]` y property sources con `server.port=8082`, JDBC `localhost:15434/db_pagatu_orden` y Swagger habilitado.

En PROD el JSON debe mostrar `server.port=9082`, host `db-pagatu-prod`, usuario `pagatu_prod`, `springdoc.swagger-ui.enabled=false` y Actuator restringido a `health,info`.

[INSERTAR CAPTURA DE PANTALLA AQUÍ: Debe mostrar la pantalla completa sin recortar, con el reloj del sistema (fecha y hora) y el nombre de usuario de Windows/Vs Code visible]

[INSERTAR CAPTURA DE PANTALLA AQUÍ: Debe mostrar la pantalla completa sin recortar, con el reloj del sistema (fecha y hora) y el nombre de usuario de Windows/Vs Code visible]

2.4. orden-ms iniciando como Config Client y `/actuator/health`

En la bitácora de arranque debe observarse la importación

`optional:configserver:http://localhost:18888` y la carga del perfil `dev`. El servicio queda en el puerto **8082** (valor que ya no está en el JAR, sino en `orden-ms-dev.yml`).

```
curl http://localhost:8082/actuator/health
curl http://localhost:8082/api/v1/ordenes/info
```

Health esperado: `status: UP` con detalle de disco y base de datos PostgreSQL cuando el contenedor `pagatu-postgres-orden` está saludable.

[INSERTAR CAPTURA DE PANTALLA AQUÍ: Debe mostrar la pantalla completa sin recortar, con el reloj del sistema (fecha y hora) y el nombre de usuario de Windows/Vs Code visible]

[INSERTAR CAPTURA DE PANTALLA AQUÍ: Debe mostrar la pantalla completa sin recortar, con el reloj del sistema (fecha y hora) y el nombre de usuario de Windows/Vs Code visible]

3. Comparación DEV vs PROD y comprensión técnica

3.1. Tabla comparativa `orden-ms-dev.yml` vs `orden-ms-prod.yml`

Aspecto	DEV (<code>orden-ms-dev.yml</code>)	PROD (<code>orden-ms-prod.yml</code>)
Puerto HTTP	8082	9082
Host / URL de BD	localhost:15434/db_pagatu_orden	db-pagatu-prod:5432/db_pagatu_orden
Usuario BD	postgres	pagatu_prod
Contraseña BD	postgres (laboratorio)	P4g4Tu#Prod2026
JPA <code>show-sql</code>	true	false
Nivel de logs	root: INFO, paquete propio DEBUG, SQL DEBUG	root: WARN, paquete propio INFO, SQL WARN
Swagger UI / OpenAPI	habilitados (<code>springdoc.*.enabled: true</code>)	deshabilitados (<code>false</code>)
Actuator expuesto	health,info,metrics con <code>show-details: always</code>	solo health,info con <code>show-details: never</code>

Aspecto	DEV (<code>orden-ms-dev.yml</code>)	PROD (<code>orden-ms-prod.yml</code>)
Flyway	habilitado	habilitado (mismo contrato de esquema, distinto runtime)

3.2. Valores que antes estaban hardcodedos y ahora están externalizados

En un `application.yml` local de Sesión 1 (o en perfiles `application-dev.yml` / `application-prod.yml` empaquetados en el JAR) solían ir hijos:

- `server.port`
- `spring.datasource.url`, `username` y `password`
- `spring.jpa.show-sql` y niveles `logging.level.*`
- exposición de Actuador (`management.endpoints.web.exposure.include`)
- interruptores de Swagger (`springdoc.swagger-ui.enabled`)

Tras la migración a Config Client, el módulo `pagatu-orden-ms` **solo declara** `spring.application.name=orden-ms`, el perfil activo y:

```
spring.config.import:
  "optional:configserver:${CONFIG_SERVER_URL:http://localhost:18888}"
```

No existen `application-dev.yml` ni `application-prod.yml` en el cliente. Puerto, JDBC, logs, Flyway operativo, Actuador y Swagger se resuelven en caliente desde `infra/pagatu-config/config-repo`. Cambiar PROD (por ejemplo desactivar Swagger o apuntar a otro host de PostgreSQL) **no exige recompilar** `pagatu-orden-ms`.

4. Explicación del patrón Centralized Configuration

El patrón **Centralized Configuration** separa el **artefacto inmutable** (código + dependencias) de las **propiedades de ambiente**. `pagatu-config`, anotado con `@EnableConfigServer`, no ejecuta reglas de negocio: sirve documentos YAML por HTTP.

Con perfil `native`, el backend no es Git sino el filesystem (`search-locations`). El cliente `orden-ms` no incrusta DEV/PROD: al arrancar, Spring Boot procesa `spring.config.import` y solicita `{name}/{profile}` → `orden-ms/dev`. El servidor lee `orden-ms-dev.yml`, arma un `Environment` (property sources + `version/label`) y lo serializa. El `Environment` del cliente fusiona esas claves con mayor prioridad que el YAML mínimo del classpath.

Consecuencia operativa: un cambio de puerto, credencial o flag de Swagger se edita en `config-repo` y se refleja en la siguiente (o en un refresh, si se habilita). **No hay recompilación ni nuevo JAR** por un ajuste de ambiente. En un sistema con muchos microservicios, un solo Config Server evita N copias divergentes de `application-prod.yml` y reduce el riesgo de desplegar un binario “de desarrollo” a producción.

`optional`: permite arrancar el cliente si el servidor aún no está arriba (útil en el laboratorio); en producción el equipo puede quitar `optional` o usar `fail-fast` para no subir instancias “mudas” sin configuración.

5. Registro de error o hallazgo técnico

Problema. Al primer arranque se usó `spring.application.name: pagatu-orden-ms` (nombre del directorio Maven) mientras los archivos del repo se llamaban `orden-ms-dev.yml` y `orden-ms-prod.yml`. El Config Server respondía `http://localhost:18888/pagatu-orden-ms/dev` con un Environment **vacío** (sin `server.port` ni `datasource`). El cliente quedaba en el puerto 8080 por defecto, no conectaba a PostgreSQL en 15434 y Actuator/Health fallaba o arrancaba sin las propiedades del laboratorio.

Diagnóstico. Se contrastó la convención `{application}-{profile}.yml` con el `name` publicado en `/orden-ms/dev` (JSON con property source `file:./config-repo/orden-ms-dev.yml`) frente a `/pagatu-orden-ms/dev` (sin ese source). En logs de `org.springframework.cloud.config` no aparecía la ubicación esperada para `pagatu-orden-ms`.

Solución aplicada. Se fijó `spring.application.name: orden-ms` en `services/pagatu-orden-ms/src/main/resources/application.yml`, alineado con los YAML del config-repo. Se validó `GET /orden-ms/dev` y el arranque en **8082**. Hallazgo colateral: si `search-locations: file:./config-repo` se ejecuta con CWD distinto a `infra/pagatu-config`, el servidor no encuentra los YAML; se documentó ejecutar Maven desde ese directorio o definir `CONFIG_REPO_LOCATION`.

6. Reflexión técnica

Config Server escala el gobierno de la plataforma cuando PagaTú deja de ser un JAR y pasa a decenas de microservicios e instancias. En lugar de duplicar `application-prod.yml` en catálogo, órdenes, pagos y notificaciones —con el riesgo de que un equipo actualice el host de PostgreSQL y otro no—, hay **una sola fuente** por aplicación y perfil. La coherencia DEV/PROD deja de depender de “acordarse de no commitar secretos en el código”: el binario es el mismo y el ambiente lo inyecta el servidor. El ciclo de vida se acorta porque un cambio de flag (Swagger off, nivel de log, endpoint de Actuator) es un commit en config-repo y un reinicio o refresh, no un pipeline de build completo. A más réplicas de `orden-ms`, todas leen el mismo documento; se elimina la deriva de configuración, se audita quién cambió PROD y se gana agilidad sin sacrificar el control académico y operativo de los ambientes.

7. Respuestas a las preguntas de defensa (sección 4.5)

1. ¿Qué problema resuelve Spring Cloud Config Server en PagaTú?

Centraliza puerto, JDBC, logs, Flyway y Actuator/Swagger por ambiente, para no recompilar cada microservicio cuando cambia DEV o PROD.

2. ¿Por qué el perfil `native` y `search-locations: file:./config-repo`?

En el laboratorio el backend es el filesystem local: el servidor lee YAML sin Git. `native` activa ese repositorio; `search-locations` apunta a `config-repo`.

3. ¿Cómo se elige el archivo que recibe `orden-ms`?

Por `spring.application.name` + perfil activo. Con `name=orden-ms` y `dev` se sirve `orden-ms-dev.yml` en `GET /orden-ms/dev`. Si el `name` no coincide, el Environment llega vacío.

4. ¿Qué cambia entre DEV y PROD además del puerto?

PROD usa otro host y credenciales de PostgreSQL, logs más silenciosos, Swagger apagado y Actuator mínimo sin detalles de health, reduciendo superficie de ataque y ruido.

5. ¿Para qué sirve `optional:configserver:...` en el cliente?

Importa la configuración remota al arrancar. El prefijo `optional:` evita que falle el bootstrap si Config Server aún no está en `18888`; el valor se parametriza con `CONFIG_SERVER_URL`.

Anexo A — Mapa de archivos S02

```
infra/pagatu-config/
  pom.xml
  src/main/java/pe/edu/upeu/config/PagatuConfigApplication.java
  (@EnableConfigServer)
  src/main/resources/application.yml                (puerto 18888,
native)
  config-repo/
    pagatu-catalogo-ms-dev.yml
    pagatu-catalogo-ms-prod.yml
    orden-ms-dev.yml
    orden-ms-prod.yml

services/pagatu-catalogo-ms/                (Config Client; sin application-
dev/prod.yml)
services/pagatu-orden-ms/                  (trabajo autónomo; name = orden-ms)

docker-compose.s02.yml                      (PostgreSQL 15432 catálogo / 15434 órdenes)
```

Anexo B — Cómo generar el PDF de entrega

1. Completar el **Link de GitHub** y pegar las capturas en los placeholders de la sección 2.
 2. Exportar este Markdown a PDF con el nombre **S02_Equipo01_CuadrosJhosef.pdf**.
 3. Verificar que cada captura muestre reloj del sistema y usuario de Windows/VS Code.
-

Fin del informe individual S02

Cuadros Flores Jhosef Giampiere · Equipo 1 · Gestión Centralizada de Configuración y Ambientes